

5.5 Windowed Point-Addition Architecture

In this section, we combine windowing with the parallel accumulation architecture of Section 5.2 and Montgomery’s trick in Section 5.4. The point-addition resources presented below are obtained from our gate-level implementations, and the QROM resources are computed analytically from the coherent unary-iteration construction. The final resource values are obtained by combining these module costs.

We use the same parallelism parameter w as in Section 5.2 and introduce one additional parameter s , the scalar-window size. In this setting, w is the number of parallel window-accumulation lanes. Increasing s reduces the number of point additions but increases the QROM table size exponentially, while increasing w reduces the number of sequential point-addition layers but requires more qubits.

5.5.1 Window Decomposition and Shifted Tables

Let $m = n + 1$ be the size of each exponent register and let

$$\ell = \left\lceil \frac{m}{s} \right\rceil, \quad L_s = 2\ell.$$

We decompose the two exponents as

$$k = \sum_{j=0}^{\ell-1} k_j 2^{sj}, \quad l = \sum_{j=0}^{\ell-1} l_j 2^{sj}.$$

The j -th window contains

$$b_j = \min\{s, m - sj\}$$

bits. Consequently, its table size is $K_j = 2^{b_j}$. In particular, if m is not divisible by s , the most significant window is shorter than s and must not be charged as a 2^s -entry table.

For each $S \in \{P, Q\}$ and window index j , we classically generate a shifted affine-point table

$$\tilde{\mathcal{T}}_j^S[u] = [u2^{sj}]S + \Delta_j^S, \quad 0 \leq u < 2^{b_j},$$

where Δ_j^S is a fixed, classically generated curve point. We choose the offset by rejection sampling so that the table contains no point at infinity. These fixed offsets only translate the final group element and do not change the hidden subgroup. Looking up and summing all P - and Q -window entries gives

$$F'_s(k, l) = [k]P + [l]Q + \Delta_{\text{total}}, \quad \Delta_{\text{total}} = \sum_{S \in \{P, Q\}} \sum_{j=0}^{\ell-1} \Delta_j^S.$$

The reported windowed circuit implements the shifted oracle

$$|k, l\rangle |0\rangle \rightarrow |k, l\rangle |F'_s(k, l)\rangle.$$

No final addition of $-\Delta_{\text{total}}$ is performed. Translation by the fixed point Δ_{total} is a bijection and hence $F'_s(k, l) = F'_s(k', l')$ is equivalent to $[k]P + [l]Q = [k']P + [l']Q$. Therefore, the shifted oracle has the same collision relation and hidden subgroup as the original double-scalar-multiplication oracle, and it gives the same probability distribution after the quantum Fourier transform. We do not include a final subtraction of Δ_{total} in the resource estimates below.

5.5.2 Coherent Unary-Iteration QROM

For a table $\mathcal{T}$ containing $K = 2^b$ affine points, write the u -th entry as $\mathcal{T}[u] = (x_u, y_u)$. Let $W_u = \text{enc}(x_u) \parallel \text{enc}(y_u)$ be the corresponding $2n$ -bit classical word. The QROM implements

$$U_{\mathcal{T}} : |u\rangle |r\rangle \rightarrow |u\rangle |r \oplus W_u\rangle.$$

Here, the two coordinates are represented in the polynomial basis for Karatsuba arithmetic and in the same L -basis as the multiplication circuit for AC-based arithmetic. We further define $h_u = \text{HW}(W_u)$. Thus, $0 < h_u \leq 2n$, and h_u depends on the table entry and its basis representation.

We use a unary-iteration selector [1] together with balanced CNOT fanout for the data writes. A b -bit lookup uses the original b -bit window as its address, $b - 1$ clean selector ancillas, a $2n$ -qubit output register, and $2n - 1$ reusable fanout ancillas. The selector qubits are computed using Toffoli gates in a binary tree structure. A negative control is implemented by applying X gates before and after the corresponding Toffoli gate. At a leaf, the active selector controls the CNOTs that write the nonzero bits of W_u to the output register.

For one coherent lookup, the Toffoli count before local cancellations is

$$T_{\text{QROM}} = 2(K - 2).$$

With balanced fanout, its CNOT count is

$$C_{\text{QROM}} = (K - 2) + \sum_{u=0}^{K-1} (3h_u - 2).$$

The first term comes from transitions in the unary selector tree. For one table row, $h_u - 1$ CNOTs create the balanced fanout, h_u CNOTs write the data in parallel, and another $h_u - 1$ CNOTs clear the fanout. Since every accepted affine table word is nonzero, $h_u \geq 1$.

For the gate order used in our implementation, the overall depth of one lookup under ASAP scheduling is

$$D_{\text{QROM}} = 3K - 4 + \sum_{u=0}^{K-1} (2 \lceil \log_2 h_u \rceil + 1).$$

Here, $3K - 4$ is the data-independent selector-traversal contribution. For each row, the balanced fanout, parallel data write, and fanout recovery contribute $2 \lceil \log_2 h_u \rceil + 1$ layers, with $\lceil \log_2 1 \rceil = 0$. The value of h_u depends on the table entry and its basis representation. For a table-independent upper bound, we use $h_u = 2n$ for every table row u . This assumes the maximum possible Hamming weight for every $2n$ -bit table word. For example, when $n = 163$, each row is treated as if all 326 bits were nonzero. The resulting bounds are

$$C_{\text{QROM}}^{\max} = (K - 2) + K(6n - 2) = K(6n - 1) - 2,$$

and

$$D_{\text{QROM}}^{\max} = 3K - 4 + K(2 \lceil \log_2 (2n) \rceil + 1).$$

Since $h_u \leq 2n$ for every row, these give upper bounds on the CNOT count and full depth of this QROM implementation under ASAP scheduling. The substitution $h_u = 2n$ does not change the selector Toffoli count, Toffoli depth, or QROM width.

We implement $U_{\mathcal{T}}^\dagger$ by applying the gates of $U_{\mathcal{T}}$ in reverse order on the same address register, selector ancillas, fanout ancillas, and output register. Therefore, the lookup and its inverse have the same resource cost.

For resource estimation, we combine the lookup, the computation of the two coordinate sums needed for the subsequent point addition, and the inverse lookup as one module. Adjacent self-inverse gates within this module are cancelled. For the traversal order described above, its resource costs are

$$\begin{aligned} T_{\text{module}} &= 4K - 2b - 6, \\ C_{\text{module}} &= 2C_{\text{QROM}} + 4n - 2(h_{K-1} - 1), \\ D_{\text{module}} &= 2D_{\text{QROM}} - 2b - 2 \lceil \log_2 h_{K-1} \rceil + 3. \end{aligned}$$

Here, h_{K-1} is the Hamming weight of the last table entry in the unary-iteration traversal. These formulas only use cancellations inside this module, and we do not use cancellations across different QROM and point-addition modules. In the upper-bound model, $h_{K-1} = 2n$.

5.5.3 Parallel Window Schedule

We order the L_s window descriptors as $P_0, Q_0, P_1, Q_1, \dots, P_{\ell-1}, Q_{\ell-1}$, which also places the two possibly shortened top windows last. Let $1 \leq w \leq L_s$.

In Phase I, the first w QROM outputs are used directly to initialize w separate accumulators:

$$R_i \leftarrow \tilde{T}_i[u_i], \quad 0 \leq i < w.$$

The $2n$ -bit QROM output registers of these first lookups become the accumulator registers, so they are kept until the forward oracle is reversed. Each later Phase-I batch performs one coherent lookup and one uncontrolled quantum–quantum point addition in every active lane. The control bit used in the bitwise architecture is now part of the QROM address and is not needed as a separate control on the point-addition circuit.

In Phase II, the w partial sums are combined by a binary tree of uncontrolled quantum–quantum point additions. No QROM is used in this reduction tree. After initialization, Phase I and all of Phase II use the quantum–quantum point-addition circuit in Fig. 3(b).

The numbers of sequential point-addition layers are

$$h_{\text{acc}} = \left\lceil \frac{L_s}{w} \right\rceil - 1, \quad h_{\text{tree}} = \lceil \log_2 w \rceil,$$

and the total number of point additions is

$$(L_s - w) + (w - 1) = L_s - 1.$$

Thus, windowing reduces the point-addition count from $2n + 1$ to

$$2 \left\lceil \frac{n+1}{s} \right\rceil - 1.$$

When $w = 1$, the windowed accumulation is serial. When $w = L_s$, all window points are loaded in parallel and only the binary-tree reduction remains. We keep $s = 1$ as the original bitwise baseline, which bypasses QROM and uses the controlled constant-point addition in Fig. 3(a). A literal $s = 1$ instance of the windowed construction would use a two-entry QROM followed by an uncontrolled quantum–quantum addition, so it is not the same circuit as the bitwise baseline.

5.5.4 Early QROM Uncomputation

For every post-initialization Phase-I lane, the logical operation is

$$|u\rangle |R\rangle |0\rangle \mapsto |u\rangle |R + \tilde{T}[u]\rangle |0\rangle.$$

Suppose one lane adds the looked-up point $T = (x_T, y_T)$ to the current accumulator $R = (x_R, y_R)$. Immediately after the lookup, we compute two clean n -bit registers,

$$z = x_R + x_T, \quad v = y_R + y_T.$$

The remaining point-addition circuit uses

$$\begin{aligned} \lambda &= vz^{-1}, \\ x_3 &= \lambda^2 + \lambda + z + a, \\ y_3 &= \lambda(x_R + x_3) + x_3 + y_R. \end{aligned}$$

Once z and v have been computed, the remaining circuit no longer uses x_T or y_T . We can therefore apply $U_{\mathcal{T}}^\dagger$ at this point, clearing the QROM output register, selector ancillas, and fanout ancillas.

The operations are scheduled as

$$U_{\mathcal{T}} \longrightarrow (z, v) \text{ computation} \longrightarrow U_{\mathcal{T}}^\dagger \longrightarrow \text{Montgomery inversion} \longrightarrow \text{remaining point addition}.$$

Thus, each post-initialization window addition requires one lookup and one inverse lookup, even though the second input coordinates appear several times in Fig. 3(b). The registers z and v are kept until their last uses and are then uncomputed together with the arithmetic work registers.

5.5.5 Resource Estimation and Parameter Selection

We present the resource estimates for the windowed architecture in Table 1. The point-addition costs are obtained from our gate-level AC-based implementation of the quantum–quantum point addition used in Phase II, while the QROM costs are computed from the closed-form formulas above using the conservative choice $h_u = 2n$ for every table entry. The total resources are obtained by combining the QROM and point-addition modules. We do not generate a single gate sequence for the complete windowed circuit. For depth, we do not reschedule gates across module boundaries, so the reported NCT depth is an upper bound.

We evaluate

$$2 \leq s \leq 18, \quad 1 \leq w \leq L_s,$$

and report the configurations

$$\begin{aligned} (s_T^*, w_T^*) &= \arg \min_{s, w} T(s, w), \\ (s_{\text{TDW}}^*, w_{\text{TDW}}^*) &= \arg \min_{s, w} \text{TDW}(s, w), \\ (s_{\text{DW}}^*, w_{\text{DW}}^*) &= \arg \min_{s, w} \text{DW}(s, w). \end{aligned}$$

For comparison, we include the original bitwise architecture as the $s = 1$ baseline. Our estimates use a fully coherent NCT implementation of QROM. Measurement-assisted QROM variants [2] and their resource trade-offs are left for future study.

For $n = 571$, the parameter search was still incomplete at the time of the rebuttal. We evaluated 1,734 of the 2,866 valid (s, w) configurations. The reported rows are therefore the best configurations found so far and are not claimed to be global optima.

Table 1: Resource estimates for the windowed architecture and comparison with the Balanced+AC-inv baseline.

n	Strategy/target	(s, w)	#Toffoli	#CNOT	Width	Toffoli depth	NCT depth upper bound	DW-cost	TDW-cost
163	Balanced+AC-inv	(1, 56)	6341768	857859900	512450	810	577596	1.08×2^{38}	415084500
	Window-QROM: min Toffoli	(10, 34)	859352	172273452	126944	4416	283806	1.05×2^{35}	560584704
	Window-QROM: min TDW	(6, 56)	1140472	169570892	204856	608	264086	1.57×2^{35}	124552448
	Window-QROM: min DW	(8, 27)	927068	155263818	114260	3372	289358	1.92×2^{34}	385284720
233	Balanced+AC-inv	(1, 59)	13394252	2570641706	916826	990	938002	1.56×2^{39}	907657740
	Window-QROM: min Toffoli	(10, 48)	1673988	441495408	261705	4440	386846	1.47×2^{36}	1161970200
	Window-QROM: min TDW	(5, 94)	2768248	568697540	501605	560	427272	1.56×2^{37}	280898800
	Window-QROM: min DW	(9, 32)	1739348	432758628	220895	6448	416854	1.34×2^{36}	1424330960
283	Balanced+AC-inv	(1, 64)	20232912	4693744666	1279002	1136	1306596	1.52×2^{40}	1452946272
	Window-QROM: min Toffoli	(11, 52)	2434896	862161814	352297	8568	550770	1.41×2^{37}	3018480696
	Window-QROM: min TDW	(5, 114)	4156864	1026739070	753590	596	560740	1.54×2^{38}	449139640
	Window-QROM: min DW	(8, 15)	2921228	814667288	247916	9560	731498	1.32×2^{37}	2370076960
571	Balanced+AC-inv	(1, 128)	86049284	40693341630	5402040	1352	3075488	1.89×2^{43}	7303558080
	Window-QROM: min Toffoli [†]	(11, 104)	9202596	5703468882	1472990	8688	1279034	1.71×2^{40}	12797337120
	Window-QROM: min TDW [†]	(6, 192)	14822796	7441985718	2685300	848	1395718	1.70×2^{41}	2277134400
	Window-QROM: min DW [†]	(10, 30)	10157204	6145473890	935355	29092	1730120	1.47×2^{40}	27211347660

Here s is the scalar-window size and w is the number of parallel accumulation lanes. Rows marked [†] are the best values found among the 1,734 evaluated configurations for $n = 571$ and are not claimed to be global optima.

References

- [1] R. Babbush, C. Gidney, D. W. Berry, N. Wiebe, J. McClean, A. Pater, A. Fowler, and H. Neven, “Encoding electronic spectra in quantum circuits with linear T complexity,” *Physical Review X*, vol. 8, no. 4, Art. no. 041015, 2018. [doi:10.1103/PhysRevX.8.041015](https://doi.org/10.1103/PhysRevX.8.041015).
- [2] D. W. Berry, C. Gidney, M. Motta, J. R. McClean, and R. Babbush, “Qubitization of arbitrary basis quantum chemistry leveraging sparsity and low rank factorization,” *Quantum*, vol. 3, Art. no. 208, 2019. [doi:10.22331/q-2019-12-02-208](https://doi.org/10.22331/q-2019-12-02-208).